

API v1

Integration guide

Servers. Provisioning. Access. Game administration.

<https://eserv.pl/api/v1>

- 01 Select a template and node
- 02 Validate the installation plan
- 03 Create and track the operation

Your first request

Set up authentication, select your servers, and make a request.

Create a token in Account / API tokens, assign servers and the servers.read scope. The secret is displayed only once. Store it on your integration backend.

```
# Bash / curl; load TOKEN from a secure environment
export API="https://eserv.pl/api/v1"
curl --fail-with-body -sS "$API/servers?sort=address&limit=25" \
  -H "Authorization: Bearer $TOKEN"
```

Successful responses contain data and requestId. Paginated lists also include nextCursor. Errors contain error.code, error.message and requestId. Keep the request ID in your integration logs.

Contents

- 03 Tokens and permissions
- 04 Server list and details
- 05 Templates, nodes and installation plans
- 06 Server creation and idempotency
- 07 Operations and error handling
- 08 Power and backups
- 09 Accounts and server access
- 10 AMXX administrators
- 11 Complete endpoint reference
- 12 Integration rules and contract scope

Examples use demonstration values. Replace IDs, template versions, addresses and variables with the values returned by your API.

Tokens and permissions

Token scopes and the owner's current permissions apply together.

Area	Token scopes
Servers and metrics	servers.read, resources.read
Power and backups	servers.power, backups.read, backups.create
Provisioning	templates.read, nodes.read, servers.create
Operations	operations.read
Accounts	users.read, users.create
Server access	members.read, members.write
AMXX administrators	game-admins.read, game-admins.write

Provisioning, catalog, account and membership endpoints require a panel administrator. Access to an individual server is still restricted by its assignment to the token. Roles and account status are checked on every request.

Install servers policy

A provisioning token specifies allowed nodeIds and templateIds, a maxServers creation budget, and per-server maxCpu and maxMemoryMb limits. A newly created server is assigned to that token.

The creation budget also counts failed and uncertain admissions. Deleting a server does not replenish it. The token does not automatically gain access to all future servers.

Headers and limits

```
Authorization: Bearer <TOKEN>
Content-Type: application/json
Idempotency-Key: order-20260927-000001
```

120 requests per minute per token. Respect Retry-After on 429. Idempotency-Key applies to server creation, backup creation and power actions. An API token cannot manage API tokens.

Servers and their status

GET /servers · GET /servers/{id} · GET /servers/{id}/resources

id is the canonical UUID; number is the global server number. All /servers/{id} paths accept either form, for example /servers/101.

List parameter	Meaning
sort	id (default), number, name, address; IP addresses and ports are compared numerically
search / status	Search and status filtering
game / node	Game catalog ID / node UUID or local
limit / after	1-100, default 50 / UUID from nextCursor

Keep the same filters and sort order when fetching subsequent pages. The cursor refers to the current inventory. On invalid_cursor, restart from the first page.

Key response fields

Group	Fields
Identity	id, number, name, provider, catalogId, node, template
Status	status, runtimeStatus, available, stale, observedAt, desiredState
Network and limits	ports, limits, uptimeSeconds
Installation and game	installation, game
Integration	capabilities, links; resourceUsage in details with resources.read

The list uses periodically refreshed inventory; the detail endpoint queries the runtime. available=false / stale=true / status=unknown means there is no fresh confirmation. Do not interpret it as stopped.

game describes the game response independently of the process. null means no measurement or no support. /resources returns the latest sample; when no fresh sample is available, observedAt and resources are null.

Preparing an installation

Select a published Native template and an allocated port pool.

Step	Endpoint	Result
1	GET /templates	IDs, versions, required ports and variables
2	GET /nodes	Available locations
3	GET /nodes/{id}/allocations	Configured IP addresses and TCP/UDP pools
4	POST /servers/plan	Validation of the complete request

provision.json

```
{
  "nodeId": "NODE_UUID",
  "templateId": "TEMPLATE_ID",
  "templateVersion": 2,
  "name": "New server",
  "resourceLimits": { "cpu": 1, "memoryMb": 1024 },
  "bindings": [
    { "key": "game", "hostIp": "192.0.2.10", "host": "auto" }
  ],
  "variables": {}
}
```

This illustrates the request structure, not a ready-to-use configuration for every game. Use the current template version, all required bindings and its variables. The address 192.0.2.10 is a demonstration value.

```
curl --fail-with-body -sS "$API/servers/plan" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  --data-binary @provision.json
```

A plan does not reserve ports. Creation rechecks availability under a lock. The auto value selects a port from the assigned pool; linked TCP/UDP bindings share the same IP address and port.

Creating a server

POST /servers · servers.create + provisioning policy

```
curl --fail-with-body -sS -D admission.headers \  
"$API/servers" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-H "Idempotency-Key: order-20260927-000001" \  
--data-binary @provision.json
```

202 Accepted confirms admission of the task. The Location header points to /api/v1/operations/{id}. Store the operation ID and the key before proceeding with your integration workflow.

One order, one key

Situation	Action
Same key and payload	The same operation is returned; Idempotency-Replayed: true
Same key, different payload	409; do not change an order that has already been submitted
Timeout or uncertain result	Check the operation. Keep the key; do not create a new order
New, independent installation	Use a new persistent key and a separate order record

Keys contain 16-128 characters: letters, digits, hyphens or underscores. Installation keys use a namespace separate from backup/power.

What a completed installation means

The files and container are prepared, the number comes from the central database, and the game server remains stopped. Starting it is a separate, explicit power action.

Public provisioning supports published Native templates only. Requests cannot supply custom scripts, images, mount paths or startup commands.

Operations and errors

GET /operations · GET /operations/{id} · operations.read

The operation list belongs to the token, is ordered newest first, and shows the admission state. Read the current result and progress through links.self or Location. List limit: 1-100, default 50; after accepts the cursor UUID.

Installation state	Interpretation
dispatching / accepted	The request is being dispatched or has been accepted
uncertain	Dispatch result is unconfirmed; check the same operation
running	Installation is in progress; use stage and percent
completed	Installation is complete; the game remains stopped
failed	Installation failed; inspect the result before a new attempt

Poll at a moderate interval, for example every 3-5 seconds, and increase it for long operations. This is a client recommendation, not a protocol requirement.

Client responses

HTTP	Handling
400	Correct the payload or cursor; do not retry unchanged
401 / 403	Check the token, role and permissions
404	The resource does not exist or is no longer accessible to the token
409	Distinguish a conflict, stale ETag and uncertain admission
428	Include a current If-Match when editing game administrators
429	Wait for the duration specified by Retry-After
5xx / no response	Check the operation result; preserve the idempotency key

409 with data.status=uncertain does not mean it is safe to submit a new installation. The panel does not automatically redispach an uncertain request.

Power and backups

Server control and backups are asynchronous operations.

Start, stop and restart

```
curl --fail-with-body -sS "$API/servers/101/power" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-H "Idempotency-Key: power-20260927-000001" \
--data '{"action":"start"}
```

Allowed actions: start, stop, restart. Requires servers.power and the token owner's server.power permission. Completion confirms execution of the action; check game readiness separately in server details.

Listing backups

GET /servers/{id}/backups requires backups.read and the owner's corresponding permission. Results are sorted by name; after accepts a backup name. Limit: 1-100, default 50.

Backup field	Meaning
name / kind	Name and type: file or directory
sizeBytes	Size or null
modifiedAt	Modification time or null

Creating a Native backup

```
curl --fail-with-body -sS "$API/servers/101/backups" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-H "Idempotency-Key: backup-20260927-000001" \
--data '{"name":"Before-update"}
```

Requires backups.create on both the token and owner. The response is asynchronous and includes Location. The name is optional; the public contract does not expose archive restoration, deletion or download.

Accounts and access

Panel administrators manage standard user accounts and server assignments.

Creating an account

```
POST /api/v1/users
Authorization: Bearer <TOKEN>
Content-Type: application/json

{"username":"client01","password":"<PASSWORD_8_TO_128_CHARS>"}
```

Requires users.create; returns 201. Usernames contain 1-12 letters, digits, dots, underscores or hyphens. The account always receives the user role. A duplicate username returns 409.

GET /users with users.read returns id, username, role and enabled. Limit: 1-100; after is a numeric ID. If account creation returns an uncertain response, look up the unique username instead of creating another account.

Assigning a server

```
PUT /api/v1/servers/101/members/USER_ID
Authorization: Bearer <TOKEN>
Content-Type: application/json

{"preset":"server-admin"}
```

Preset	Access
viewer	Read the console
console-operator	Console, game commands and power
server-admin	The above plus files, backups, schedules and SFTP

No preset grants terminal access, CPU/IP/port editing or startup environment editing. PUT replaces membership; DELETE removes it. GET /servers/{id}/members returns members and presets.

Changes require members.write, a panel administrator role and the server assigned to the token. They apply to standard user accounts, not operators or superadmins. SFTP access does not imply a public password-issuance endpoint.

AMXX administrators

CS 1.6 / ReHLDS · file version control with ETag

1. Read entries and their version

```
curl --fail-with-body -sS -D admins.headers \  
"$API/servers/101/game-admins" \  
-H "Authorization: Bearer $TOKEN"
```

The response contains provider=amxx, entries, unmanaged and version. The ETag header contains the quoted version hash. GET requires game-admins.read and the owner's fs.read permission.

2. Add or update an administrator

```
curl --fail-with-body -sS -X PUT \  
"$API/servers/101/game-admins/STEAM_0:1:123" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-H 'If-Match: "CURRENT_HASH_FROM_ETAG"' \  
--data '{"flags":"bcdefiju}"'
```

Use the exact ETag value, including quotation marks. Mutations require game-admins.write and fs.read and fs.write. Flags contain 1-25 letters a-y. DELETE on the same path removes the entry and also requires If-Match.

Result	Next step
Write successful	Keep the new version; the response may include snapshotId and warning
409 - stale version	Fetch the list again, reconcile the change and use the new ETag
428	A valid If-Match is missing

A write creates a snapshot and replaces the file atomically. Comments and custom entries are preserved; passwords are not exposed. An unsupported authentication format blocks modification of that entry.

The API does not automatically run an administrator reload command. SourceMod and CS2 adapters are outside the current contract.

Endpoint reference

All paths below are relative to /api/v1.

Method	Path	Scope
GET	/servers	servers.read
GET	/servers/{id}	servers.read
GET	/servers/{id}/resources	resources.read
GET	/templates	templates.read
GET	/nodes	nodes.read
GET	/nodes/{id}/allocations	nodes.read
POST	/servers/plan	servers.create
POST	/servers	servers.create
GET	/operations	operations.read
GET	/operations/{id}	operations.read
POST	/servers/{id}/power	servers.power
GET	/servers/{id}/backups	backups.read
POST	/servers/{id}/backups	backups.create
GET	/users	users.read
POST	/users	users.create
GET	/servers/{id}/members	members.read
PUT / DELETE	/servers/{id}/members/{userId}	members.write
GET	/servers/{id}/game-admins	game-admins.read
PUT / DELETE	/servers/{id}/game-admins/{steamId}	game-admins.write

A scope is necessary but not sufficient. The role, current owner permissions, server assignment and any provisioning policy still apply.

Contract and integration rules

Documentation baseline: 27 September 2026 / revision 6c2adf2.

Rules to implement in your client

Store the UUID as the technical identifier; display the global number to users.

Store the idempotency key and operation ID with the order. A timeout does not prove that the operation was not executed.

Separate container status from the game response. Do not convert unknown or null to stopped or 0.

Read capabilities and respect current permissions; access can be revoked while the integration is running.

Handle pagination, Retry-After and ETag conflicts. Do not log tokens or passwords.

Outside API v1 in this release

Legacy LinuxGSM/OVHcloud provisioning, operator creation, account deletion and password resets, SourceMod/CS2 administrators, public writes for addons/configuration/schedules, SFTP password issuance, and multi-recipient integration webhooks.

Contract sources

Repository file	Purpose
docs/pro/openapi-v1.json	Request/response schemas and API paths
docs/pro/API-V1-GUIDE.md	Behavior and integration workflows

This PDF describes the deployed contract. Examples contain no real secrets or guaranteed template IDs. Fetch the current catalog and validate a plan before installation.

Release verification

295 backend tests and 5 UI tests. Production checks covered API reads and an installation plan. Actual creation of a new server through the API was not part of the production verification for this release.